

GeoRoute

Pan-India Offline Map & Routing Webapp

Beginner Codebase Guide

Every function explained + end-to-end flows

Written for beginners who want to understand how this project works.

Read top-to-bottom: big picture first, then each file and function,
then the step-by-step flows that connect the UI to the database.

Stack: Angular 17 + OpenLayers | Spring Boot 3.2 (Java 17)
PostgreSQL + PostGIS | GeoTools 29.2

Output file: GeoRoute_Beginner_Codebase_Guide.pdf

Table of contents

- 1. What is GeoRoute? (plain English)
- 2. Mental model: roads as a graph
- 3. Technology stack (what each piece does)
- 4. Folder map (where to look)
- 5. How to run the project
- 6. Big-picture architecture
- 7. End-to-end flows (with diagrams)
- 8. Database schema explained
- 9. Backend: every class and function
- 10. Frontend: every class and function
- 11. Scripts, SQL, and packaging
- 12. REST API cheat sheet
- 13. Configuration keys
- 14. Suggested learning path
- 15. Glossary

1. What is GeoRoute? (plain English)

GeoRoute is a web app that shows a map of India roads and lets you pick a start point and an end point. It then draws the best driving route (and optional alternate routes) between those points.

Unlike Google Maps, this app is designed to work OFFLINE after setup. The map images come from a local GeoTIFF file. The roads come from a local Shapefile loaded into PostgreSQL/PostGIS. Routing (A* or Dijkstra) is written in Java inside the backend.

What a user does

- Open the map in the browser.
- Click once for the start (source). The click snaps to the nearest road.
- Click again for the destination. That also snaps to a road.
- Choose A* or Dijkstra, choose how many alternate routes (K), click Find Routes.
- Colored lines appear on the map with distance and travel time.

2. Mental model: roads as a graph

Before reading code, learn this one idea: roads are stored as a GRAPH.

- Node (RoadNode): a point where roads meet or a road endpoint. Has id, longitude, latitude.
- Edge (RoadEdge): a directed road segment from one node to another. Has length, speed, cost, geometry.
- Cost: usually travel time in seconds (length / speed). Shorter cost = better route.
- Path: an ordered list of nodes and edges from source to destination.
- A* / Dijkstra: algorithms that find the cheapest path through the graph.

When you click the map, the app does NOT use your exact click as a graph node. It SNAPS the click to the nearest road edge, then picks the nearer endpoint node. Routing always runs between two node IDs.

Click (lat, lng)

- > Snap to nearest road edge (PostGIS)
- > Choose nearer endpoint nodeId
- > Find path(s) between sourceNode and destNode
- > Convert edge geometries into a polyline for the map

3. Technology stack (what each piece does)

Layer	Tool	Beginner meaning
UI	Angular 17	Framework for the web page and buttons
Map	OpenLayers 7	Draws the map, pins, and route lines
API	Spring Boot 3	Java server that answers /api/* requests
GIS Java	GeoTools	Reads Shapefile + GeoTIFF files
Geometry	JTS	Java geometry objects (points, lines)
Database	PostgreSQL	Stores nodes and edges permanently
Spatial DB	PostGIS	Adds map queries (nearest road, bbox)
Build BE	Maven	Builds the Java backend
Build FE	npm / Angular CLI	Builds / serves the frontend

4. Folder map (where to look)

```

Maps_shrushru_DRD0/
  README.md          <- project overview + run instructions
  data/              <- roads Shapefile + GeoTIFF basemap
  db/init.sql        <- creates PostGIS tables
  georoute-backend/  <- Java Spring Boot server
    src/main/java/com/georoute/
      GeoRouteApplication.java <- program entry
      config/ controllers/ dto/ model/
      repository/ routing/ service/ util/
  georoute-frontend/ <- Angular UI
    src/app/components/ <- map, route panel, toast
    src/app/services/   <- HTTP + shared state
    src/app/models/     <- TypeScript interfaces
  scripts/             <- packaging + doc generators
  offline-bundle/      <- portable one-folder deploy

```

Tip: Start reading from GeoRouteApplication.java (backend) and main.ts + map.component.ts (frontend). Those are the 'doors' into the app.

5. How to run the project

One-time setup

- Install JDK 17, Maven, Node 18/20, Angular CLI 17, PostgreSQL + PostGIS.
- Run setup.bat (Windows) or ./setup.sh (Linux/macOS).
- Create DB: createdb georoute then psql -d georoute -f db/init.sql
- Put road Shapefile + roads_basemap.tif into data/.
- Set DB password in georoute-backend/.../application.properties.

Daily run (development)

```

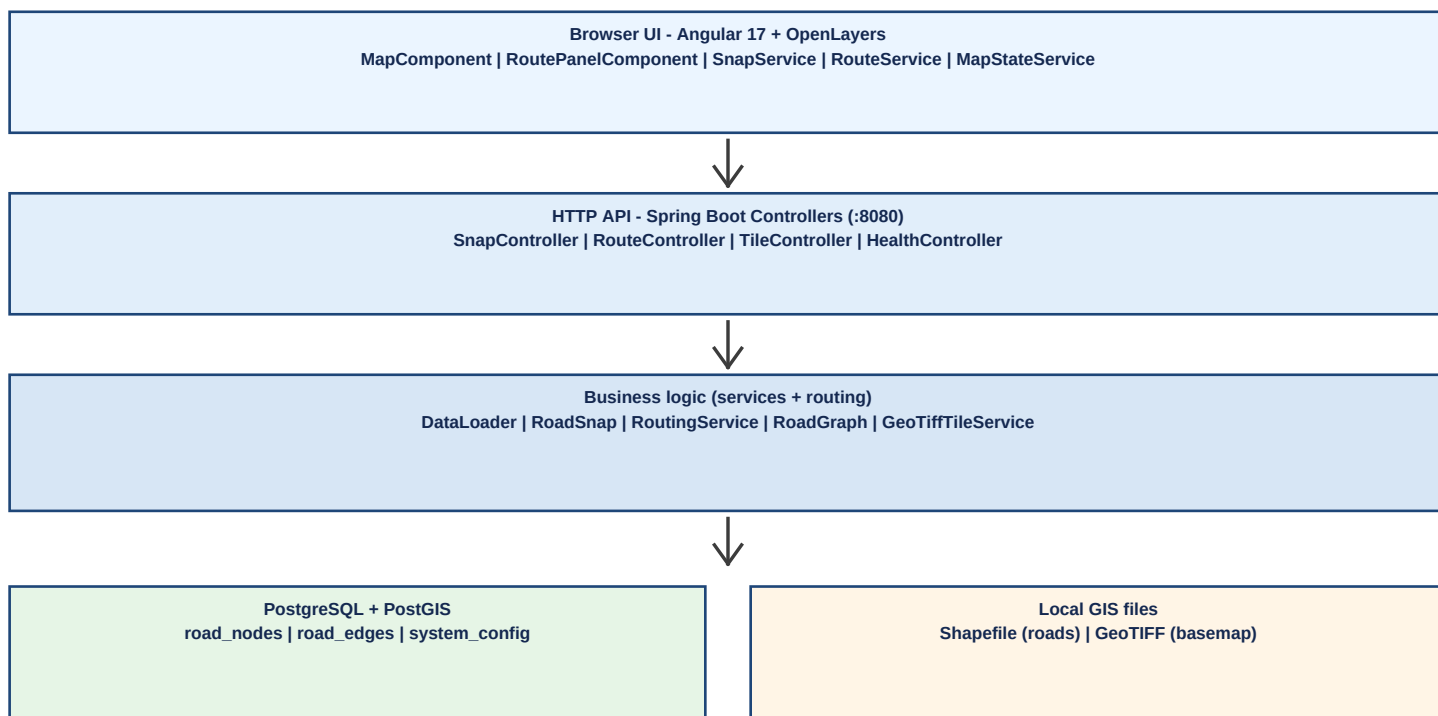
# Terminal 1 - backend
cd georoute-backend
mvn spring-boot:run          # http://localhost:8080

# Terminal 2 - frontend
cd georoute-frontend
npm start                    # http://localhost:4200
                             # proxies /api -> :8080

```

First backend start may take a long time while the Shapefile is ingested into PostGIS. Later starts skip ingest if system_config already says shp_loaded=true.

6. Big-picture architecture

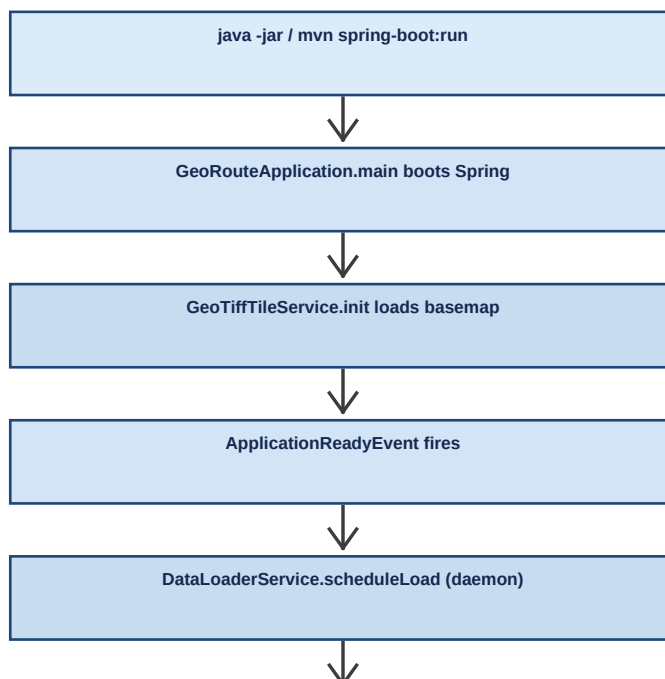


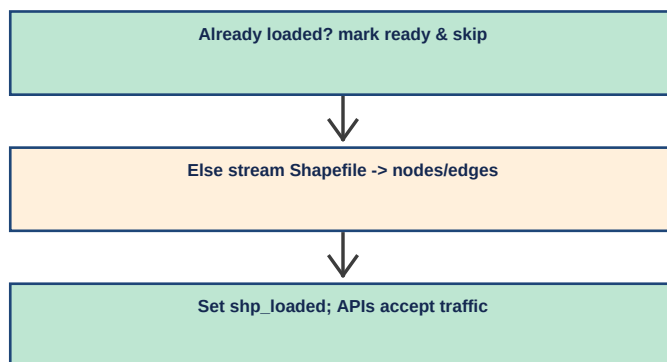
Request direction: UI calls `/api/*` -> Controllers validate/receive JSON -> Services do work -> Repositories/JDBC talk to PostGIS OR GeoTools reads files -> JSON/PNG returns to UI.

7. End-to-end flows (with diagrams)

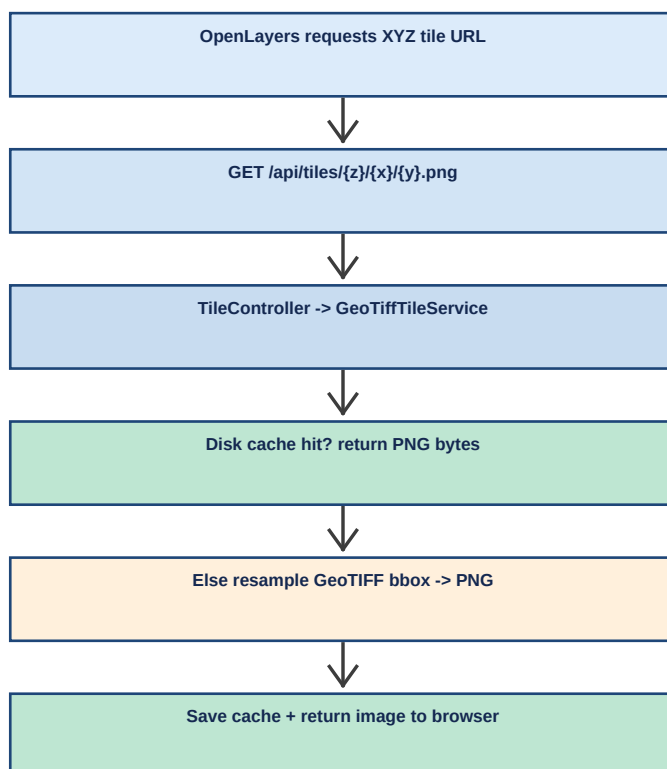
These flows are the 'story' of the app. Every function later in this PDF fits into one of them.

7.1 Startup + Shapefile ingest

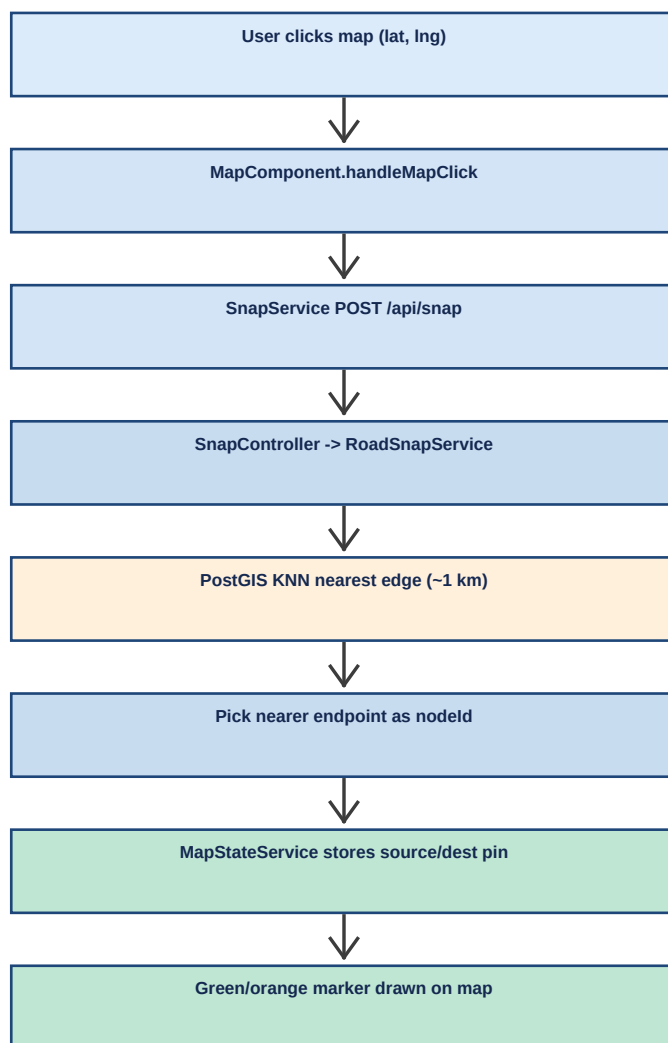




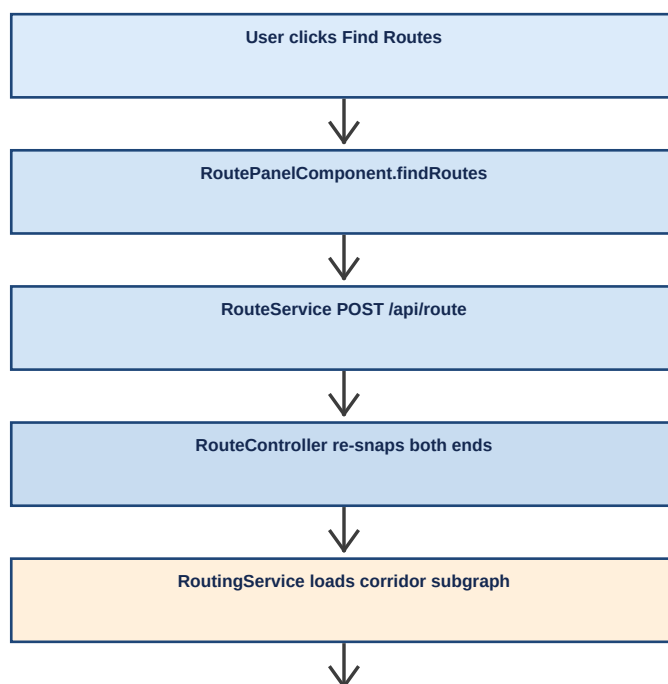
7.2 Map tiles (pan / zoom)

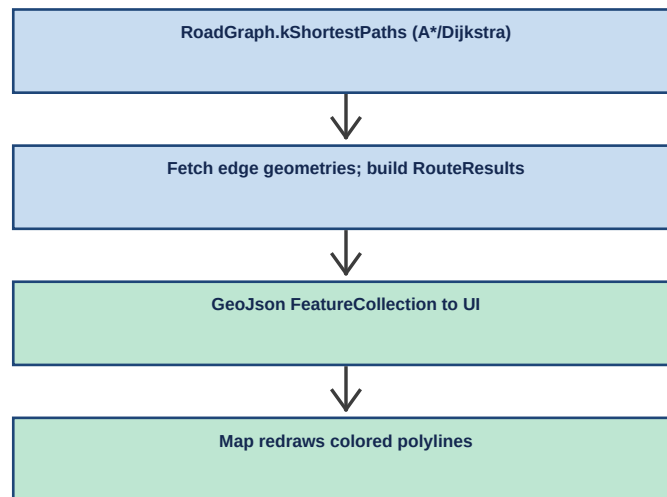


7.3 Click map -> snap pin

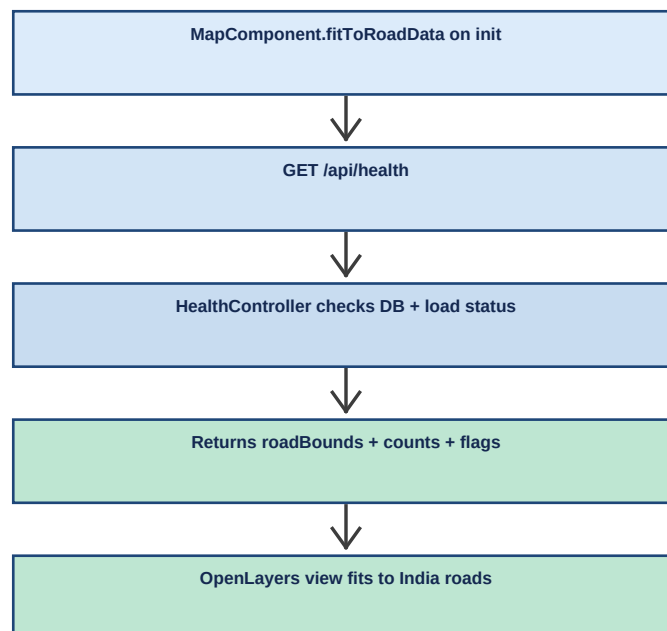


7.4 Find Routes





7.5 Health + fit map bounds



7.6 Inside routing (corridor + K paths)

Routing does NOT load all of India's roads into memory for every request. That would be too slow/huge. Instead:

- Compute a bounding box (envelope) around source and destination.
- Add padding (`paddingDegrees`). Longer trips get more padding / hierarchy filtering.
- SQL loads only nodes/edges inside that corridor into a `RoadGraph`.
- Find the best path with A* (or Dijkstra).
- For alternates: penalize used edges and search again; keep paths that are diverse enough (Jaccard).
- Stitch edge `LineString` geometries into one coordinate list for GeoJSON.

```

sourceNode + destNode + k + algorithm
-> loadSubgraph(bbox +/- padding [, major roads for long trips])
-> RoadGraph.kShortestPaths(...)
    path1 = shortestPath
    applyPenalty(path1 edges)
    path2 = search again with penalties...
    keep if overlap with previous paths is low enough
-> fetchGeometries(edgeIds)
-> List<RouteResult> -> GeoJSON
  
```

8. Database schema explained

Created by db/init.sql. You should understand three tables:

system_config

- Key/value flags. Important key: shp_loaded = whether roads were ingested.

road_nodes

- id: primary key
- geom: Point in EPSG:4326 (longitude/latitude)
- coord_key: quantized lon/lat string used to deduplicate nodes during ingest

road_edges

- id, source_node, target_node
- geom: LineString of the road segment
- length_m, maxspeed_kmh, cost_seconds, oneway, name, fclass (road type)

GIST indexes on geometries make 'nearest road' and 'edges in bbox' queries fast. Without PostGIS, snapping and corridor loading would be much harder.

9. Backend: every class and function

Package root: com.georoute under georoute-backend/src/main/java/com/georoute/. Below, each function is explained in beginner terms.

9.1 GeoRouteApplication.java (entry point)

This is where the Java program starts.

main(String[] args)

Parameters: Command-line args (usually unused)

Returns: void

What it does: Starts the Spring Boot application.

How (simple): Sets GeoTools axis order to XY (lon/lat) so GIS files are not swapped, then calls SpringApplication.run.

9.2 config/CorsConfig.java

Allows the Angular app on another port to call the API during development.

addCorsMappings(CorsRegistry registry)

Parameters: Spring CORS registry

Returns: void

What it does: Registers which browser origins may call /api/**.

How (simple): Reads cors.allowed-origins from application.properties (e.g. localhost:4200).

9.3 config/PostGISConfig.java

geometryFactory()

Parameters: none (Spring @Bean)

Returns: GeometryFactory

What it does: Creates a shared JTS GeometryFactory for WGS84 (EPSG:4326).

How (simple): Other beans can inject this when they need to build Point/LineString objects.

9.4 config/SpaWebConfig.java

Used in the offline JAR so one server can serve both UI files and API.

addViewControllers(ViewControllerRegistry registry)

Parameters: View controller registry

Returns: void

What it does: Maps '/' to index.html so opening the site loads the Angular app.

addResourceHandlers(ResourceHandlerRegistry registry)

Parameters: Resource handler registry

Returns: void

What it does: Serves static Angular files from classpath:/static/ and falls back to index.html for SPA routes.

How (simple): Important: paths starting with api/ are NOT treated as SPA files.

9.5 controller/RouteController.java

RouteController(RoutingService, RoadSnapService)

Parameters: Injected services

Returns: controller instance

What it does: Constructor: stores routing and snap services for the /api/route endpoint.

route(RouteRequest request)

Parameters: JSON body with source/dest lat/Ing, numAlternatePaths, algorithm

Returns: ResponseEntity (GeoJSON or error)

What it does: Main routing API handler.

How (simple): 1) Snap source and dest. 2) Call RoutingService.route. 3) Wrap results as GeoJSON FeatureCollection. Handles DataNotReady / NoRoadNearby / generic errors with proper HTTP status codes.

9.6 controller/SnapController.java

SnapController(RoadSnapService)

Parameters: Snap service

Returns: controller instance

What it does: Constructor injection for snap endpoint.

snap(SnapRequest request)

Parameters: JSON { lat, lng }

Returns: ResponseEntity (SnapResult or SnapError)

What it does: Snaps a map click to the nearest road node.

How (simple): Delegates to RoadSnapService.snap; returns 404 if no road nearby, 503 if data not ready.

9.7 controller/TileController.java

TileController(GeoTiffTileService)

Parameters: Tile service

Returns: controller instance

What it does: Constructor for tile endpoint.

tile(int z, int x, int y)

Parameters: Zoom z and tile indices x,y

Returns: ResponseEntity<byte[]> PNG

What it does: Returns one map tile image.

How (simple): Asks GeoTiffTileService for PNG bytes; sets Content-Type image/png and Cache-Control no-store.

9.8 controller/HealthController.java

HealthController(JdbcTemplate, DataLoadStatus)

Parameters: DB access + load status bean

Returns: controller instance

What it does: Constructor for health checks.

health()

Parameters: none

Returns: Map<String,Object> JSON

What it does: Reports whether PostGIS works, whether roads are loaded, counts, bounds, and loading message.

How (simple): Frontend uses roadBounds to zoom the map to the road data extent.

9.9 dto/ (request and response shapes)

RouteRequest.java

- Fields: sourceLat, sourceLng, destLat, destLng, numAlternatePaths, algorithm.
- Lombok @Data generates getters/setters. This is the body of POST /api/route.

SnapRequest.java

- Fields: lat, lng. Body of POST /api/snap.

SnapError.java

- Fields: error, message. Returned when snap fails.

GeoJson.java

fromRoutes(List<RouteResult> routes)

Parameters: List of computed routes

Returns: Map representing a GeoJSON FeatureCollection

What it does: Converts internal RouteResult objects into GeoJSON the map can draw.

How (simple): Each route becomes a Feature with LineString geometry and properties (distance, time, rank).

9.10 model/ (domain objects)

RoadNode.java

- id, lng, lat. One graph vertex.

RoadEdge.java

- id, sourceId, targetId, lengthMeters, maxspeedKmh, costSeconds, oneway, name, fclass, etc.
- Represents a directed connection used by the router.

RoadEdge(... 10-arg constructor ...)

Parameters: All edge fields

Returns: RoadEdge

What it does: Builds an edge object when reading from JDBC/SQL results.

SnapResult.java

- snappedLat, snappedLng, nodeId, roadName (and related fields). Result of snapping.

RouteResult.java

- One finished route: coordinates, distance, duration, rank/index.

9.11 repository/RoadNodeRepository.java

findAll()

Parameters: none

Returns: List<RoadNode>

What it does: Loads every node from the database.

How (simple): Used for smaller/test scenarios; production routing uses corridor SQL in RoutingService instead.

count()

Parameters: none

Returns: long

What it does: Counts rows in road_nodes.

9.12 repository/RoadEdgeRepository.java

findAllDirected()

Parameters: none

Returns: List<RoadEdge>

What it does: Loads edges and expands bidirectional roads into two directed edges.

How (simple): If oneway is false, creates both A->B and B->A.

fetchGeometries(Collection<Long> edgeIds)

Parameters: Set/list of edge IDs used by a path

Returns: Map<edgeId, list of [lng,lat]>

What it does: Fetches the actual road shapes for drawing.

How (simple): Uses ST_AsGeoJSON in SQL, then parseLineStringCoords.

parseLineStringCoords(String geoJson)

Parameters: GeoJSON geometry text

Returns: List<double[]> coordinates

What it does: Parses a LineString's coordinates array from GeoJSON text.

count()

Parameters: none

Returns: long

What it does: Counts road_edges rows.

9.13 routing/Algorithm.java

fromString(String value)

Parameters: "astar" or "dijkstra" (case-insensitive)

Returns: Algorithm enum

What it does: Converts the UI/API string into an enum. Defaults to ASTAR if unknown.

9.14 routing/Path.java

Path(List nodes, List edges, double cost)

Parameters: Node ids, edge objects, total cost

Returns: Path

What it does: Holds one complete route through the graph.

lengthMeters()

Parameters: none

Returns: double

What it does: Sums the length of all edges in the path.

signature()

Parameters: none

Returns: String

What it does: Builds a string key from the node sequence (to detect duplicate paths).

concat(Path root, Path spur)

Parameters: Root prefix path + spur continuation

Returns: Path

What it does: Joins two path pieces (used by classic Yen's algorithm in JdbcRoadGraph).

How (simple): Avoids duplicating the junction node.

9.15 routing/RoadGraph.java (ACTIVE router)

This is the production in-memory graph used by RoutingService. Think of it as a city map loaded into RAM for one request's corridor.

RoadGraph(nodes, edges, maxSpeedKmh)

Parameters: Node map, directed edges, max speed for heuristic

Returns: RoadGraph

What it does: Builds adjacency lists: for each node, the outgoing edges.

nodeCount() / hasNode(id)

Parameters: optional node id

Returns: int / boolean

What it does: How many nodes are in the subgraph; whether a node exists.

edgeKey(RoadEdge e)

Parameters: An edge

Returns: String

What it does: Unique key for a directed edge (used with penalty maps).

shortestPath(source, target, algorithm)

Parameters: Start node, end node, ASTAR or DIJKSTRA

Returns: Optional<Path>

What it does: Public API to find one best path.

How (simple): Calls search() with empty penalties.

search(source, target, algorithm, penalties)

Parameters: Ends + algo + optional edge cost multipliers

Returns: Optional<Path>

What it does: Best-first search using a priority queue ($f = g + h$).

How (simple): g = cost so far. h = heuristic (haversine/maxSpeed for A*, 0 for Dijkstra). Penalties make previously used edges more expensive so alternate routes avoid them.

heuristic(node, target, algorithm)

Parameters: Current node, goal, algorithm

Returns: double

What it does: Optimistic estimate of remaining travel time.

How (simple): A*: distance/maxSpeed. Dijkstra: always 0 (becomes uniform-cost search).

reconstruct(cameFromNode, cameFromEdge, source, target, cost)

Parameters: Parent maps from search

Returns: Path

What it does: Walks backward from target to source to rebuild the node/edge lists.

kShortestPaths(source, target, k, algorithm)

Parameters: Ends, how many paths, algorithm

Returns: List<Path>

What it does: Finds up to K diverse routes.

How (simple): Repeatedly search, then applyPenalty on used edges, then keep a path only if maxJaccardOverlap with previous paths is below a threshold.

applyPenalty(path, penaltyMap, factor)

Parameters: A found path, mutable map, multiplier

Returns: void

What it does: Increases cost of edges already used so the next search prefers different roads.

maxJaccardOverlap(candidate, accepted)

Parameters: New path vs already kept paths

Returns: double

What it does: Measures similarity of edge sets (Jaccard index). High = too similar, discard.

undirectedEdgeIds(path)

Parameters: A path

Returns: Set<Long>

What it does: Collects edge IDs for overlap comparison.

Inner class NodeState

NodeState.compareTo

Parameters: Another NodeState

Returns: int

What it does: Orders priority-queue entries by f-score (g+h) so the best candidate is expanded first.

9.16 routing/JdbcRoadGraph.java (ALTERNATE, unused)

Same idea as RoadGraph, but neighbors are loaded from the database on demand, and K-paths use classic Yen's algorithm. It is NOT wired into RoutingService today; kept as an alternate design. Knowing it exists helps if you explore experiments.

JdbcRoadGraph(JdbcTemplate, maxSpeed)

Parameters: DB access + heuristic speed

Returns: JdbcRoadGraph

What it does: Constructs a lazy DB-backed graph.

nodeCount / hasNode / loadNode / outgoing

Parameters: node ids as needed

Returns: counts / boolean / RoadNode / list of edges

What it does: Lazy accessors: ask Postgres for a node or its outgoing edges when needed.

shortestPath / search / heuristic / reconstruct

Parameters: Same roles as RoadGraph

Returns: Optional<Path> / Path / double

What it does: A* or Dijkstra over DB-fetched neighbors.

kShortestPaths(...)

Parameters: s, t, k, algorithm

Returns: List<Path>

What it does: Classic Yen's K-shortest paths (spur nodes + root prefixes).

samePrefix(path, root)

Parameters: Candidate path and root prefix

Returns: boolean

What it does: Helper for Yen: checks whether a path starts with a given root sequence.

9.17 service/RoutingService.java

Orchestrates one routing request: load corridor, run RoadGraph, attach geometries.

RoutingService(RoadEdgeRepository, JdbcTemplate)

Parameters: Edge repo + JDBC

Returns: service

What it does: Constructor injection.

route(sourceNodeId, targetNodeId, k, algorithm)

Parameters: Two node IDs, number of paths, algorithm

Returns: List<RouteResult>

What it does: Top-level routing method called by RouteController.

How (simple): Computes padding, loadSubgraph, kShortestPaths, fetchGeometries, buildCoordinates for each path.

paddingDegrees(source, target, longTrip)

Parameters: Endpoint nodes + whether trip is long

Returns: double

What it does: How much to expand the search bbox in degrees.

How (simple): Far-apart points need a wider corridor so the graph still connects.

loadSubgraph(source, target, padding, useHierarchy)

Parameters: Ends, pad, hierarchy flag

Returns: RoadGraph

What it does: SQL-selects edges/nodes inside the envelope and builds an in-memory RoadGraph.

How (simple): For long trips, prefer major road classes plus buffers near endpoints so the subgraph stays small.

loadNodes(Set<Long> ids)

Parameters: Node id set

Returns: Map<Long, RoadNode>

What it does: Loads node coordinates in chunks (avoids huge IN clauses).

buildCoordinates(path, geoms, sourceSnap, destSnap)

Parameters: Path + geometry map + snap ends

Returns: List of [lng,lat]

What it does: Stitches each edge's LineString into one continuous polyline for the map.

How (simple): Uses appendSegment; falls back to fallbackSegment if geometry missing; may reverse segments.

fallbackSegment(edge, ends)

Parameters: Edge + endpoint coords

Returns: coordinate list

What it does: Straight line between nodes if DB geometry is missing.

reversed(segment)

Parameters: List of coordinates

Returns: Reversed list

What it does: Flips a segment so direction matches travel.

appendSegment(out, seg)

Parameters: Output list + next segment

Returns: void

What it does: Appends points, skipping a duplicate join point.

invalidateGraph()

Parameters: none

Returns: void

What it does: No-op placeholder (graphs are per-request now).

loadNode(long id)

Parameters: Node id

Returns: RoadNode or null

What it does: Fetch a single node row.

9.18 service/RoadSnapService.java

RoadSnapService(JdbcTemplate, DataLoadStatus)

Parameters: DB + readiness flags

Returns: service

What it does: Constructor.

snap(double lat, double lng)

Parameters: Click coordinates

Returns: SnapResult

What it does: Finds nearest road within about 1 km and returns snapped point + nodeId + road name.

How (simple): Uses PostGIS KNN (<->) / distance. Chooses the nearer of the edge's two endpoints as the graph node. Throws DataNotReadyException or NoRoadNearbyException when appropriate.

9.19 service/GeoTiffTileService.java

init()

Parameters: none (called at startup)

Returns: void

What it does: Opens the GeoTIFF basemap with GeoTools and prepares for tile rendering.

getTile(z, x, y)

Parameters: XYZ tile indices

Returns: byte[] PNG

What it does: Returns a tile from disk cache or renders it.

renderTile(z, x, y)

Parameters: XYZ indices

Returns: BufferedImage

What it does: Computes the geographic bbox of the tile and resamples the GeoTIFF into a 256x256 image.

paintCoverage(image, coverage envelope...)

Parameters: Target image + raster coverage

Returns: void

What it does: Copies RGB or grayscale samples into the tile image.

clampByte(double v)

Parameters: Sample value

Returns: int 0..255

What it does: Clamps a pixel sample into a valid byte range.

toPng(BufferedImage image)

Parameters: Rendered image

Returns: byte[]

What it does: Encodes the image as PNG bytes for HTTP.

tile2lon / tile2lat

Parameters: tile x or y + zoom

Returns: degrees

What it does: Converts Web Mercator tile indices to WGS84 longitude/latitude.

9.20 service/DataLoaderService.java

Loads the OSM roads Shapefile into PostGIS once (streaming, batched).

scheduleLoad()

Parameters: none (ApplicationReadyEvent)

Returns: void

What it does: Starts a background daemon thread for ingest so the server can finish booting.

loadInBackground()

Parameters: none

Returns: void

What it does: Idempotent ingest: skip if loaded, else stream shapefile, on failure mark status failed.

isLoaded()

Parameters: none

Returns: boolean

What it does: Reads system_config for shp_loaded.

clearRoadData()

Parameters: none

Returns: void

What it does: Truncates node/edge tables (used when force-reload is enabled).

countNodes() / countEdges()

Parameters: none

Returns: long

What it does: Row counts for logging/status.

ensureCoordKeyColumn()

Parameters: none

Returns: void

What it does: Makes sure road_nodes.coord_key exists/unique for dedup.

ingestStreaming(File shp)

Parameters: Shapefile path

Returns: long[] counts

What it does: Reads features one-by-one with GeoTools and inserts nodes/edges in batches.

buildSegments(lineString, attributes...)

Parameters: Geometry + OSM attrs

Returns: void

What it does: Splits a road LineString into point-to-point segments and queues edges.

segmentWkt(c1, c2)

Parameters: Two coordinates

Returns: WKT string

What it does: Builds LINESTRING WKT for insert.

snap(x,y) / coordKey(x,y)

Parameters: lon/lat

Returns: quantized values / string key

What it does: Rounds coordinates so nearly-identical points share one node id.

resetSequences()

Parameters: none

Returns: void

What it does: Fixes PostgreSQL BIGSERIAL sequences after bulk load.

stringAttr(feature, name)

Parameters: Shapefile feature + attribute name

Returns: String

What it does: Safe attribute read (null-tolerant).

parseOneway(String raw)

Parameters: OSM oneway flag text

Returns: boolean

What it does: Interprets yes/true/1/-1 style oneway values.

speedForClass(String fclass)

Parameters: Road class like motorway, residential

Returns: double km/h

What it does: Default speeds when Shapefile has no maxspeed.

Inner NodeIdResolver**resolve(x, y)**

Parameters: Coordinates

Returns: long nodeId

What it does: LRU cache + upsert: get existing node by coord_key or insert a new one.

Inner EdgeBatcher

add(...) / flush()

Parameters: Edge fields / none

Returns: void

What it does: Buffers INSERT statements and flushes in batches for speed.

9.21 service/DataLoadStatus.java**isLoading / isReady / getMessage**

Parameters: none

Returns: boolean / String

What it does: Read current ingest status for APIs.

startLoading()

Parameters: none

Returns: void

What it does: Mark ingest as in progress.

finishLoading(nodes, edges)

Parameters: counts

Returns: void

What it does: Mark ready after successful ingest.

finishSkipped(nodes, edges)

Parameters: counts

Returns: void

What it does: Mark ready when data was already present.

failLoading / failMissingShapefile

Parameters: error info

Returns: void

What it does: Mark failed states with messages.

9.22 service exceptions

- DataNotReadyException: roads still loading or failed -> typically HTTP 503.
- NoRoadNearbyException: click too far from any road -> typically HTTP 404.

9.23 util/GeoUtils.java**haversineMeters(lat1, lon1, lat2, lon2)**

Parameters: Two WGS84 points

Returns: distance in meters

What it does: Great-circle distance between two lat/lon points.

How (simple): Used by A* heuristic and anywhere approximate ground distance is needed.

9.24 Tests: RoadGraphTest.java**buildGraph / addBidirectional / find**

Parameters: tiny helper graph

Returns: graph / edges / nodes

What it does: Build a small diamond graph for unit tests.

astarAndDijkstraFindIdenticalShortestPath

Parameters: none

Returns: void (JUnit)

What it does: Asserts A* and Dijkstra agree on shortest path.

kPathsReturnsDistinctAlternates

Parameters: none

Returns: void

What it does: Asserts K-path search returns diverse routes.

dijkstraEqualsAstarCostForAllPairs

Parameters: none

Returns: void

What it does: All-pairs cost equivalence on the tiny graph.

10. Frontend: every class and function

Root: georoute-frontend/src/. Angular bootstraps AppModule; the map is the main UI.

10.1 main.ts

platformBrowserDynamic().bootstrapModule(AppModule)

Parameters: none

Returns: Promise

What it does: Starts the Angular application in the browser.

How (simple): This is the frontend entry point (like main() in Java).

10.2 environments

- environment.ts (dev): apiBaseUrl and tileUrl for local proxy.
- environment.prod.ts: production tile URL (same-origin /api/tiles/...).

10.3 app.module.ts / app-routing.module.ts / app.component

- AppModule: declares MapComponent, RoutePanelComponent, ToastComponent; imports HttpClient, forms, etc.
- AppRoutingModule: routing scaffold (single-page app; little/no extra routes).
- AppComponent: shell layout that hosts map + side panel + toast.

10.4 models/

- snap-result.model.ts: TypeScript interface matching SnapResult JSON.
- route-result.model.ts: interfaces for route FeatureCollection / features / properties.

10.5 services/map-state.service.ts

Shared app state using RxJS BehaviorSubjects (pins, routes, algorithm, toasts).

showToast(message, durationMs)

Parameters: Text + optional duration

Returns: void

What it does: Pushes a toast message for ToastComponent to show, then clears it.

clearAll()

Parameters: none

Returns: void

What it does: Resets source pin, dest pin, and routes (Clear Map button).

Other state fields (typical): sourcePin\$, destPin\$, routes\$, algorithm\$, numRoutes\$, routing\$ - components subscribe and update these instead of passing props through many layers.

10.6 services/snap.service.ts

snap(lat, lng)

Parameters: Click coordinates

Returns: Observable<SnapResult>

What it does: HTTP POST to /api/snap.

How (simple): MapComponent subscribes; on success it stores the pin in MapStateService.

10.7 services/route.service.ts

findRoutes(source, dest, k, algorithm)

Parameters: Coords + K + algo string

Returns: Observable<RouteResponse>

What it does: HTTP POST to /api/route with RouteRequest body.

How (simple): RoutePanelComponent calls this when Find Routes is clicked.

10.8 components/map/map.component.ts

ngOnInit()

Parameters: Angular lifecycle

Returns: void

What it does: Creates the map, fits to data bounds, starts subscriptions.

initMap()

Parameters: none

Returns: void

What it does: Creates OpenLayers Map with XYZ tile layer pointing at /api/tiles/... and a click handler.

fitToRoadData()

Parameters: none

Returns: void

What it does: Calls /api/health and fits the view to returned roadBounds.

handleMapClick(event)

Parameters: OpenLayers click event

Returns: void

What it does: Converts click to lon/lat, calls SnapService, sets source then destination pin.

How (simple): Shows toast on errors (no road nearby, still loading).

subscribeToPins() / redrawPins()

Parameters: none

Returns: void

What it does: Watches pin state and redraws green/orange markers on a vector layer.

makePinFeature(coord, color)

Parameters: Coordinate + style color

Returns: Feature

What it does: Builds a styled point feature for a pin.

subscribeToRoutes() / redrawRoutes()

Parameters: none

Returns: void

What it does: Watches routes\$ and draws each route as a colored LineString (primary vs alternates).

ngOnDestroy()

Parameters: lifecycle

Returns: void

What it does: Unsubscribes / disposes map resources to avoid memory leaks.

10.9 components/route-panel/route-panel.component.ts

canFindRoutes (getter)

Parameters: none

Returns: boolean

What it does: True only when both pins exist and a request is not already in flight.

setAlgorithm(algo)

Parameters: "astar" | "dijkstra"

Returns: void

What it does: Updates algorithm\$ in MapStateService.

decRoutes() / incRoutes()

Parameters: none

Returns: void

What it does: Changes K (number of paths) within 1..10.

findRoutes()

Parameters: none

Returns: void

What it does: Calls RouteService with current pins/K/algorithm; on success updates routes\$; on error shows toast.

clearMap()

Parameters: none

Returns: void

What it does: Delegates to MapStateService.clearAll().

formatDistance / formatTime

Parameters: meters or seconds

Returns: display string

What it does: Human-friendly km and minutes for the results list.

10.10 components/toast/toast.component.ts

Subscribes to the toast message in MapStateService and displays a temporary notification (errors, hints like 'click destination'). Template/styles are in the matching .html/.scss files.

11. Scripts, SQL, and packaging

11.1 db/init.sql

- Enables postgis extensions.
- Creates system_config, road_nodes, road_edges + indexes.

11.2 scripts/rebuild_road_basemap.py**load_jdbc()**

Parameters: reads application.properties

Returns: dict

What it does: Parses DB URL/user/password for psycopg2.

main()

Parameters: none

Returns: void

What it does: Queries road geometries and rasterizes an RGB GeoTIFF basemap.

burn(...)

Parameters: geometries + raster arrays

Returns: void

What it does: Draws road pixels onto the raster.

11.3 scripts/generate_tech_doc.py / generate_beginner_doc.py

- generate_tech_doc.py: detailed technical PDF (architecture/ops).
- generate_beginner_doc.py: THIS beginner function+flow guide.

11.4 Packaging

- package-offline.bat / scripts/package-offline.ps1: build fat JAR, copy data, tile-cache, run scripts into offline-bundle/.
- run-offline.bat: start portable server on :8080 (UI+API together).

12. REST API cheat sheet

Method	Path	Purpose
POST	/api/snap	Snap lat/lng to nearest road node
POST	/api/route	Compute K routes between two points
GET	/api/tiles/{z}/{x}/{y}.png	Basemap PNG tile
GET	/api/health	DB/data readiness + roadBounds

Example snap body

```
{ "lat": 28.61, "lng": 77.21 }
```

Example route body

```
{
  "sourceLat": 28.61, "sourceLng": 77.21,
  "destLat": 28.70, "destLng": 77.10,
  "numAlternatePaths": 3,
  "algorithm": "astar"
}
```

13. Configuration keys

File: georoute-backend/src/main/resources/application.properties

Key	Meaning
spring.datasource.*	PostgreSQL JDBC URL/user/password
geotiff.base.path	Path to roads_basemap.tif
shapefile.base.path	Path to roads .shp
shapefile.force-reload	true = re-ingest on startup
shapefile.node-cache.size	LRU size during ingest
shapefile.attr.*	Attribute names in the Shapefile
tiles.cache.dir / enabled	PNG tile disk cache
routing.max.speed.kmh	A* heuristic divisor
server.port	Usually 8080
cors.allowed-origins	Browser origins allowed to call API

14. Suggested learning path

1. Read Section 2 (graph mental model) and Section 7 flows.
2. Click through the running UI once: snap two points, find routes.
3. Read frontend: map.component.ts -> snap.service.ts -> route-panel -> route.service.ts.
4. Read backend controllers in the same order: SnapController, RouteController.
5. Read RoadSnapService, then RoutingService, then RoadGraph.search/kShortestPaths.
6. Skim DataLoaderService + db/init.sql to see where nodes/edges come from.

- 7. Skim GeoTiffTileService to see how map images are made.
- 8. Only then open JdbcRoadGraph (unused alternate) if curious.

When debugging: check /api/health first (is data ready?). Then verify snap works. Only then debug routing. Most 'no route' issues are empty corridors, missing data, or pins not snapped.

15. Glossary

Term	Meaning
Shapefile	Common GIS file set (.shp/.dbf/.shx/.prj) for vector roads
GeoTIFF	Georeferenced image used as the basemap
PostGIS	PostgreSQL extension for spatial SQL
Snap	Move a click onto the nearest road
Node / Edge	Graph point / graph connection
A*	Shortest path using distance heuristic
Dijkstra	Shortest path without heuristic (h=0)
K-paths	Several alternate routes, not just one
Jaccard	Overlap score between two sets of edges
XYZ tiles	Map images addressed by zoom/x/y
GeoJSON	JSON format for geographic features
Corridor / bbox	Rectangle around S-T used to load a subgraph
EPSG:4326	Lon/lat coordinate system (WGS84)
SPA	Single Page App (Angular UI)
Fat JAR	One Java file with app + dependencies

Closing note

You do not need to memorize every function. Remember the four verbs of this system: LOAD roads, RENDER tiles, SNAP clicks, ROUTE paths. Every class exists to support one of those verbs. Follow the flow diagrams, then jump to the matching function sections when you need detail.

Regenerate this PDF anytime with:

```
python scripts/generate_beginner_doc.py
```